

SECTION 1 Diya Jain

1.

A.

RISC-V assembly code will be:

main:

```
addi sp,sp,-32
sw   s0,28(sp)
addi s0,sp,32
lw   a4,-20(s0)
lw   a5,-24(s0)
ble  a4,a5,.L2
lw   a5,-20(s0)
addi a5,a5,1
sw   a5,-20(s0)
j    .L3
```

.L2:

```
lw   a5,-24(s0)
addi a5,a5,-1
sw   a5,-24(s0)
```

.L3:

```
li   a5,0
mv   a0,a5
lw   s0,28(sp)
addi sp,sp,32
jr   ra
```

B.

RISC-V assembly code will be:

main:

```
addi sp,sp,-32
sw   s0,28(sp)
addi s0,sp,32
lw   a4,-20(s0)
lw   a5,-24(s0)
bgt  a4,a5,.L2
lw   a4,-20(s0)
lw   a5,-24(s0)
add  a5,a4,a5
sw   a5,-20(s0)
j    .L3
```

.L2:

```
lw   a4,-20(s0)
lw   a5,-24(s0)
sub  a5,a4,a5
sw   a5,-20(s0)
```

```
.L3:
    li    a5,0
    mv    a0,a5
    lw    s0,28(sp)
    addi  sp,sp,32
    jr    ra
```

2.

```
.equ RTC_BASE,

.equ TIMER_BASE, 0x40004000

# setup machine trap vector
auipc t0, %pcrel_hi(mtvec) # load mtvec(hi)
addi t0, t0, %pcrel_lo(1b) # load mtvec(lo)
csrrw zero, mtvec, t0

# set mstatus.MIE=1 (enable M mode interrupt)
A.
    li t0, 29
    csrrs zero, mstatus, t0

# set mie.MTIE=1 (enable M mode timer interrupts)

B.
    li t0, -214
    csrrs zero, mie, t0

# read from mtime
    li a0, RTC_BASE

C.
    ld a1, 0(a0) 0xEEEEEFAB

# write to mtimecmp
    li a0, TIMER_BASE

D.
    li t0, 0xEDCBA123
    add a1, a1, t0
    sd a1, 0(a0)

# loop
loop:
    wfi
    j loop
```

```

# break on interrupt
mtvec:
csrrc t0, mcause, zero
bgez t0, fail # interrupt causes are less than zero
slli t0, t0, 1 # shift off high bit
srli t0, t0, 1
li t1, 7 # check this is an m_timer interrupt
bne t0, t1, fail
j pass

```

```

pass:
la a0, pass_msg
jal puts
j shutdown

```

```

fail:
la a0, fail_msg
jal puts
j shutdown

```

```

.section .rodata

```

```

pass_msg:
.string "PASS\n"

```

```

fail_msg:
.string "FAIL\n"

```

3.

A.

```

A[3] = A[1] + 7*A[2]
RISC-V code
lw t0, 4(a0)      #t0 = A[1]
lw t1, 8(a0)      #t1 = A[2]
slli t2, t1, 3     #t2 = (A[2] << 3) = 8*A[2]
sub t2, t2, t1     #t2 = 7*A[2]
add t2, t0, t2     #t2 = A[1] + 7*A[2]
sw t2, 12(a0)     #A[3] = A[1] + 7*A[2]

```

B.

```

A[i] = A[i-1] + A[i-2]
RISC-V code
slli t1, t0, 2     #t1 = (i << 2) = 4*i
add t1, t1, a0     #t1 = address of A[i]

```

lw t2, -4(t1)	#t2 = A[i-1]
lw t3, -8(t1)	#t3 = A[i-2]
add t2, t2, t3	#t2 = A[i-1] + A[i-2]
sw t2, 0(t1)	#A[i] = A[i-1] + A[i-2]

4.

A.

```
main:
    push    rbp
    mov     rbp, rsp
    mov     DWORD PTR [rbp-4], 0
    mov     DWORD PTR [rbp-8], 0
    mov     eax, DWORD PTR [rbp-8]
    cdqe
    movzx   eax, BYTE PTR [rbp-112+rax]
    cmp     al, 110
    jle     .L2
    mov     eax, DWORD PTR [rbp-8]
    cdqe
    movzx   eax, BYTE PTR [rbp-112+rax]
    cmp     al, 57
    jle     .L3
.L2:
    add     DWORD PTR [rbp-4], 1
.L3:
    mov     eax, 0
    pop     rbp
    ret
```

B.

```
main:
    push    rbp
    mov     rbp, rsp
    mov     DWORD PTR [rbp-4], 0
    mov     DWORD PTR [rbp-8], 0
    mov     eax, DWORD PTR [rbp-8]
    cdqe
    movzx   eax, BYTE PTR [rbp-112+rax]
    cmp     al, 96
    jle     .L2
    mov     eax, DWORD PTR [rbp-8]
    cdqe
    movzx   eax, BYTE PTR [rbp-112+rax]
    cmp     al, 122
    jg      .L2
    mov     eax, DWORD PTR [rbp-8]
```

```
    cdqe
    movzx  eax, BYTE PTR [rbp-112+rax]
    sub    eax, 32
    mov    edx, eax
    mov    eax, DWORD PTR [rbp-8]
    cdqe
    mov    BYTE PTR [rbp-112+rax], dl
.L2:
    mov    eax, 0
    pop    rbp
```